AutoML for Time Series Meta-Learning - Classifier-Driven Model Selection using Sliding Windows

PHASE I REPORT

Submitted by

Rajsekar Balaji	**21011101095**
Parthiv Yagnesh Natarajan	**21011101085**
Rohan Suresh	**21011101101**

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE
&
DATA SCIENCE

DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING
SCHOOL OF ENGINEERING
SHIV NADAR UNIVERSITY CHENNAI

NOVEMBER 2024

SHIV NADAR UNIVERSITY CHENNAI

BONAFIDE CERTIFICATE

Certified that this report titled "**AutoML for Time Series Meta-Learning - Classifier-Driven Model Selection using Sliding Windows**" is the bonafide work of **Rajsekar Balaji** (Reg. No: **21011101095**), **Parthiv Yagnesh Natarajan** (Reg. No: **21011101085**), **Rohan Suresh** (Reg. No: **21011101101**) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. T. Nagarajan

Professor

Head of the Department

Department of Computer Science & Engineering

School of Engineering

Shiv Nadar University Chennai,

Kalavakkam -603110.

SIGNATURE

Prof. Shikha Marwaha

Professor

Supervisor

Department of Computer Science & Engineering

School of Engineering

Shiv Nadar University Chennai,

Kalavakkam -603110.

ABSTRACT

Automated Machine Learning (AutoML) systems have revolutionized how machine learning models are selected and tuned, but selecting the optimal model for time series forecasting remains a challenge. The diverse nature of time series data—characterized by varying trends, seasonality, and noise—makes it difficult to apply a one-size-fits-all solution. This project presents a meta-learning approach integrated with sliding windows to predict the best time series forecasting model for a given dataset. Sliding windows en- able the capture of inherent characteristics of the data, reflecting how it evolves over time and how the best model to use evolves with it. By generating sub-sequences and evalu- ating the performance of multiple forecasting models, the best model for each window is treated as a label. Window features are then extracted and used to train a classifier, which predicts the optimal model for new, unseen time series data based on its features. This approach automates the model selection process for time series forecasting in AutoML systems, enhancing the scalability and efficiency of forecasting tasks

Keywords - Meta Learning, Automated Model Selection, AutoML, AutoML for Time Series forecasting

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have provided invaluable guidance, support, and encouragement throughout the course of this project.

First and foremost, We are deeply grateful to **Dr. T. Nagarajan**, Professor and Head, Department of Computer Science and Engineering, School of Engineering, Shiv Nadar University Chennai, for his steadfast guidance and unwavering encouragement. His insights into the field of artificial intelligence and cognitive science have been instrumental in shaping the direction of this research.

We would like to extend our heartfelt thanks to our supervisor, **Prof. Shikha Marwaha**, Professor, Department of Computer Science and Engineering, School of Engineering, Shiv Nadar University Chennai, for her mentorship and commitment to the success of this project. Her expertise has been invaluable, providing us with the clarity and focus needed to tackle complex challenges and refine the methodology of this work.

We also acknowledge with gratitude the support of our family, whose understanding and encouragement have been crucial to the implementation of this project. Their unwavering belief in us has been a constant source of motivation and strength.

: **Rajsekar Balaji**	**Parthiv Yagnesh Natarajan**	**Rohan Suresh**
(21011101095)	**(21011101085)**	**(21011101101)**

TABLE OF CONTENTS

CHAPTER **PAGE NO.**

ABSTRACT **i**

LIST OF TABLES **v**

LIST OF FIGURES **vi**

1 INTRODUCTION **1**

1.1 BACKGROUND 1

1.1.1 Concept 1

1.1.2 Motivation 1

1.1.3 Objectives 2

2 LITERATURE SURVEY **3**

2.1 MODEL ALPHA 3

2.2 Dataset Exploration 4

3 METHODOLOGY **6**

3.1 Preprocessing 6

3.1.1 Yeo-Johnson Transformation 7

3.2 Feature extraction and model building 7

3.2.1 ARIMA: AutoRegressive Integrated Moving Average 9

3.2.2 Holt-Winters Exponential Smoothing (HWES) 10

3.2.3 Random Forest Classifier 11

4 EXPERIMENTS **12**

4.1 Trials 12

4.2 Best model 13

5 CONCLUSION **15**

REFERENCES **16**

LIST OF TABLES

2.1 Model Comparison: Training vs Test Accuracy 4

4.1 Initial Confusion Matrix 12

4.2 Initial Classification Report 12

4.3 Confusion Matrix after adding features 13

4.4 Classification Report after adding features 13

4.5 Confusion Matrix and combining two company's stock prices 13

4.6 Classification Report after combining two company's stock prices 14

4.7 Best Confusion Matrix 14

4.8 Best Classification Report 14

4.9 Model Comparison: Training vs Cross-Validation Accuracy 14

LIST OF FIGURES

2.1 Pipeline proposed by [1] 3

2.2 Trend of stock prcies of three companies 5

3.1 Proposed Pipeline 6

3.2 Stationary transformation 8

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND

1.1.1 Concept

Time series forecasting plays a crucial role in many fields, such as finance, energy, and healthcare, where accurate predictions of future values can significantly impact decision-making process. Traditional approaches involve a trial-and-error method of manually selecting and tuning models, which can be time-consuming and require domain expertise. AutoML (Automated Machine Learning) aims to streamline this process by automatically selecting, training and optimizing models based on the given dataset and evaluation criteria. This project utilizes a classification approach to classify the best model for the data by using sliding windows to capture the inherent characteristics between time steps in the data. We are using stock market data of the value of stocks from the year 2006 to the end of 2017. This approach enables the AutoML system to select the best possible forecasting model to use, after being trained on historical stock data of various stocks. The end goal is to develop a scalable and robust forecasting pipeline that adapts to varying stock market data.

1.1.2 Motivation

Traditional time series forecasting techniques often require significant time and expertise to manually select, tune, and evaluate modes. AutoML approaches use features derived from time series models for complete datasets. This approach may not capture the inherent properties of the data in a small time step and instead try to maximize the forecasting prediction for a large time step in the future. This method aims to capture the features within small windows and aim to forecast with good accuracy for immediate future outputs.

1.1.3 Objectives

The primary objective of this project is to develop an AutoML-based system for selecting the most suitable model for time series forecasting using a classification approach. The specific goals include:

- Implement a sliding window to capture data and extract features within a specific window size.
- Train different models on this window to capture the best model to be used within that window.
- Form a training dataset with the features of each window and their best model label.
- Use this data to train different classification models and selecting the optimal model based on accuracy and other metrics.

CHAPTER 2: LITERATURE SURVEY

Time series forecasting has been a very active area of research since the 1950s [2]. In the 1990s, the characteristics extracted from univariate time series were used to select the appropriate forecasting model for the first time [2]. The term 'meta-learning' was used for the first time in the literature of time series [3]. Meta learning refers to the process of automatically acquiring knowledge to identify the best forecasting model. It learns and understands the relationships between learning strategies and tasks, understanding the data and its characteristics and using this information to choose an appropriate model.

2.1 MODEL ALPHA

The direct inspiration of this project was from the paper [1], where they implemented a similar approach of using meta-features and training a classifier using these meta-features to predict an appropriate model. Two models - ARIMA (AutoRegressive Integrated Moving Average) and TBATS(Trignometric, Box-Cox, ARMA, Trend, and Seasonal Components) were used, and the classifier predicted one of these two models for the dataset of COVID-19.

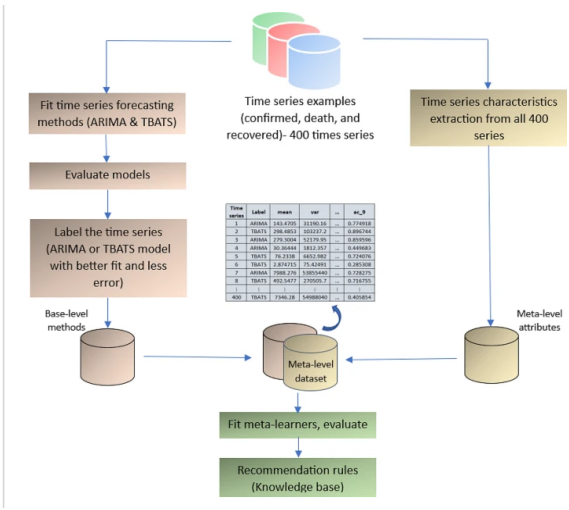


Figure 2.1: Pipeline proposed by [1]

- **Item 1**: Extracts features from the dataset
- **Item 2**: Train different models and evaluate them based on error metrics

The classifier training takes into consideration the features extracted as well as the best model label evaluated. The trained classifier is then able to select the best model to use based on given features.

Model	**Train Accuracy**	**Test Accuracy**
SVM	0.801	0.786
Random Forest	0.876	0.821
Decision Tree	**0.875**	**0.825**
ANN	0.798	0.775

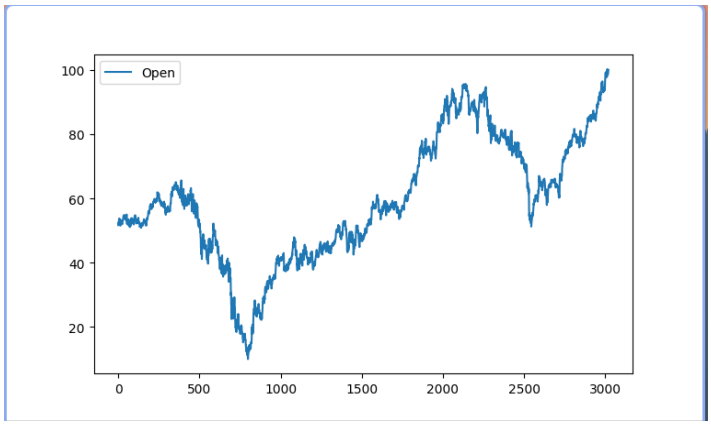
Table 2.1: Model Comparison: Training vs Test Accuracy

Based on the above results, the team concluded that Decision Trees perform the best for their given use-case.

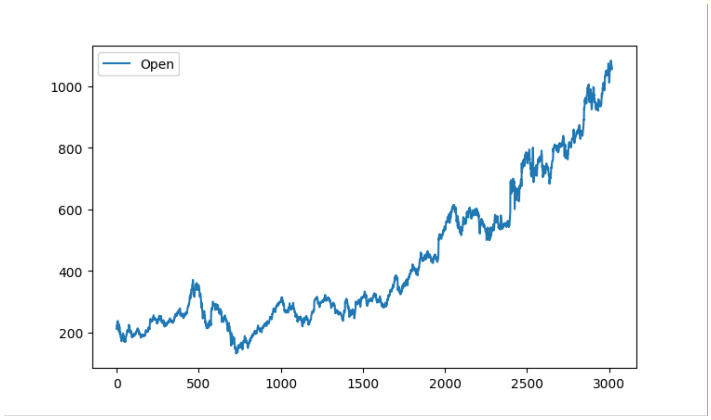
2.2 Dataset Exploration

The dataset used was acquired from kaggle [4]. All columns except the date column and the ”Open” column which contains the historical stock prices was dropped. This dataset contains historical stock market value of 30 different companies from the years 2006 to the end of 2017 (See figure 2.2)

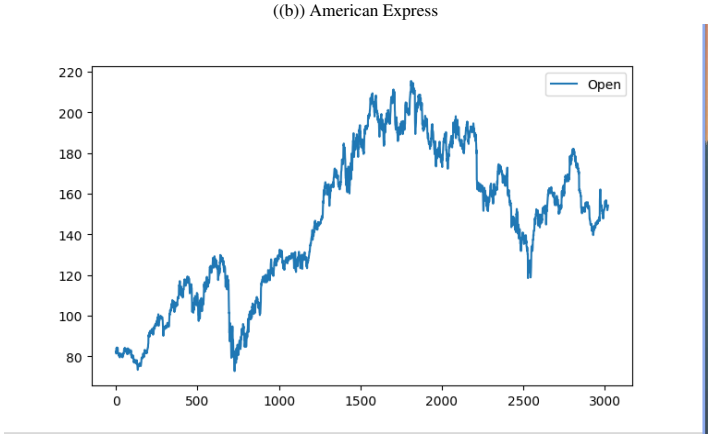
It can be seen that the dataset is seasonal with varying trends. Observations from a non-stationary time series show seasonal effects, trends, and other structures that depend on the time index. Summary statistics like the mean and variance change over time, providing a drift in the concepts a model may try to capture.



((a)) CISCO



((b)) American Express



((c)) Google

Figure 2.2: Trend of stock prcies of three companies

CHAPTER 3: METHODOLOGY

The main methodology consists of first, performing preprocessing, to make the data easier to process and run algorithms on. Second, learning best models for windows created for a specific window size. A classifier is then trained on the features extracted from a window and its best model label. The proposed model is as follows:

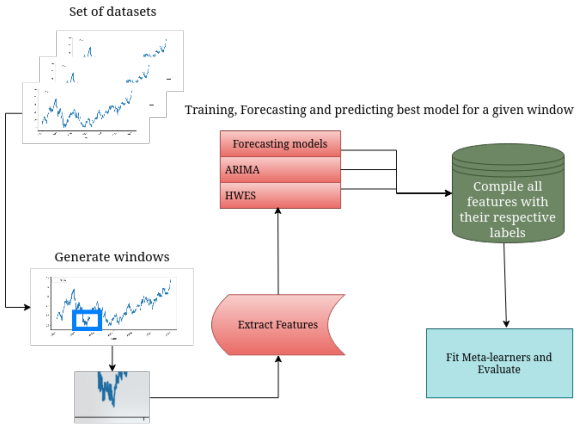


Figure 3.1: Proposed Pipeline

3.1 Preprocessing

It is easier to model time series when it is made stationary, thus the first step was making sure the datasets were made stationary. Pre-processing included removing seasonality and trends through different transformations on the data. We used log difference as a method to remove stationarity. The use of log differences is common in time series analysis, particularly when the data exhibits exponential growth or when the variance of the series increases over time. This transformation stabilizes the variance and removes trends, as noted in several studies (e.g., Hamilton, 1994 [4]; Enders, 2014 [5]).

3.1.1 Yeo-Johnson Transformation

The Yeo-Johnson transformation [5][6][7] is a power transformation technique that generalizes the Box-Cox transformation to handle both positive and negative values. It is useful for stabilizing variance and making a dataset more normal (i.e., stationary).
It is defined as:

$$y' = \begin{cases} \frac{(y+1)^{\lambda}-1}{\lambda}, & \text{if } y \geq 0 \\ -\frac{(-y+1)^{2-\lambda}-1}{2-\lambda}, & \text{if } y < 0 \end{cases}$$

Where:

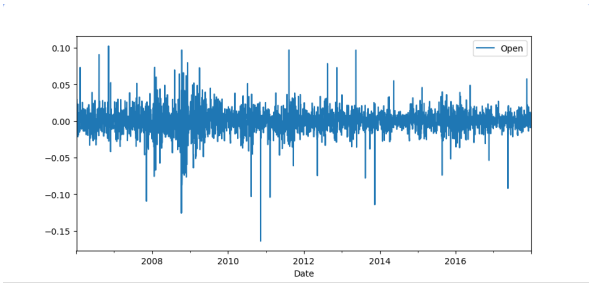
- y' is the transformed value,
- y is the original value,
- λ is the transformation parameter.

Using the Yeo-Johnson Transformation along with log-differencing made the data stationary and could be generalized to all the company specific data in the dataset.

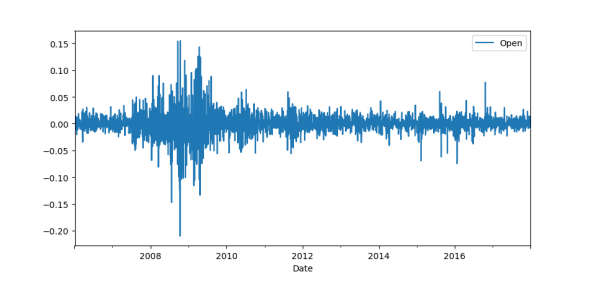
3.2 Feature extraction and model building

Window size optimization is a necessary step for making a good sliding window model. Various window sizes were tried - *200*, *100*, *120*, *150*, *50*, *20* and *75*. The best performing model used a window size of **100** with a step size of **25**, (found using trial and error). For each **100** size window, the following features are extracted:

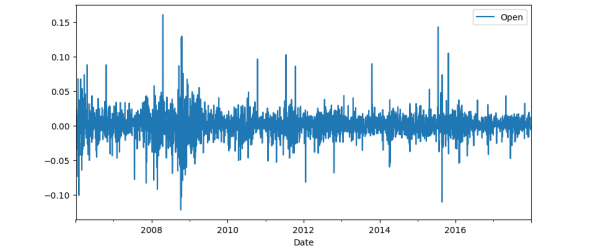
- Mean - Average of the values in the window
- Standard Deviation - Standard Deviation of the values in the window
- Maximum



((a)) CISCO



((b)) American Express



((c)) Google

Figure 3.2: Stationary transformation

- Minimum
- Range
- ADF (ADFuller)
- ACF
- Seasonal Strength - Represents the seasonality
- Skew
- Kurtosis
- First Difference - Calculates the first difference between the last two data points in

the window. It provides an indication of how much the value has changed from the second-to-last point to the last point.

- Rate of Change - Calculates the relative change between the first and last points in the window, expressed as a proportion. It is useful for understanding the overall percentage change in the data within the window.

ARIMA and HWES models were fit onto each window and the best model for each window was taken as a label. The features calculated for each window and the best model observed is coupled and used for training.

3.2.1 ARIMA: AutoRegressive Integrated Moving Average

ARIMA is a time series forecasting method that combines autoregression (AR), differencing (I), and moving averages (MA) to model a series. The model is generally represented as $ARIMA(p, d, q)$, where:

- p is the order of the autoregressive part,
- d is the degree of differencing required for stationarity,
- q is the order of the moving average part.

The ARIMA model is given by the equation:

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$$

where: - y_t is the value at time t, - ϕ_i are the coefficients of the AR part, - ϵ_t is the error term, - θ_i are the coefficients of the MA part.

ARIMA models are widely used in time series forecasting due to their simplicity and effectiveness in handling non-stationary data.
The value of the hyperparamaters *p*, *d* and *q* used here are 1, 1 and 1 respectively.

3.2.2 Holt-Winters Exponential Smoothing (HWES)

Holt-Winters Exponential Smoothing (HWES) is a popular forecasting technique used for time series data, particularly when there is seasonality. It is an extension of simple exponential smoothing that accounts for both trend and seasonality in the data. HWES is often used in situations where the data exhibits clear upward or downward trends and periodic fluctuations.

The method is composed of three components:

- Level (α): The smoothed value of the time series.
- Trend (β): The rate of change in the series over time.
- Seasonality (γ): The repeating pattern at fixed intervals.

The general form of the Holt-Winters additive model is:

$$L_t = \alpha y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

$$S_t = \gamma(y_t - L_t) + (1 - \gamma)S_{t-m}$$

$$\hat{y}_{t+h} = L_t + hT_t + S_{t+h-m}$$

where: - L_t is the level, - T_t is the trend, - S_t is the seasonal component, - $\hat{y}_{t+h}$ is the forecasted value for h-steps ahead, - α, β, and γ are the smoothing parameters, - m is the length of the seasonal cycle.

HWES is particularly useful in forecasting sales, temperature, or any other data with predictable seasonal patterns.

Utilizing both ARIMA and HWES for checking the models, creates a good balance be-

tween handling non-stationarity as well as handling seasonality. This makes sure that any residual trends in the data even after preprocessing are not taken into consideration by the classifier.

3.2.3 Random Forest Classifier

Using the features collected and the labels of ARIMA and HWES for each window taken, a classification model is trained - in this case a Random Forest Classifier (RFC). The Random Forest Classifier is an ensemble method that creates multiple decision trees and combines their outputs to improve classification accuracy. Each tree is trained on a bootstrapped sample, and at each split, a random subset of features is considered.

The final prediction is:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^{N} h_i(x)$$

where $h_i(x)$ is the prediction from the i-th tree, and N is the number of trees.

- Reduces overfitting by averaging multiple trees.
- Handles large datasets and both categorical and continuous data.

CHAPTER 4: EXPERIMENTS

The final hyperparameters like window size, and model parameters were decided on after numerous trials and experiments. Some of the results are described in the section below along with a section regarding the best performing model.

4.1 Trials

This was the initial output when no proper features were decided on, and the window size was too high for the dataset, to make enough samples.

	Predicted ARIMA	**Predicted HWES**
Actual ARIMA	4	1
Actual HWES	1	1

Table 4.1: Initial Confusion Matrix

Class	**Precision**	**Recall**	**F1-Score**	**Support**
ARIMA	0.80	0.80	0.80	5
HWES	0.50	0.50	0.50	2
Accuracy		0.71		
Macro Average	0.65	0.65	0.65	7
Weighted Average	0.71	0.71	0.71	7

Table 4.2: Initial Classification Report

We can see various issues with this initial approach, which was fixed in later iterations. The main issue being not having enough samples for both the models. Looking at the confusion matrix we see 5 samples for ARIMA and only 2 for HWES. This output was obtained after running this pipeline for 2 datasets (i.e stock prices of two companies).

Further we added more features including *seasonality* as well as *ACF* and *ADF* features. Along with this we decreased the window size to something more reasonable around 50-60

with a step size of 20, allowing us to not only get more samples, but also more accurately model the patterns.

	Predicted ARIMA	**Predicted HWES**
Actual ARIMA	11	6
Actual HWES	2	16

Table 4.3: Confusion Matrix after adding features

Class	**Precision**	**Recall**	**F1-Score**	**Support**
ARIMA	0.85	0.65	0.73	17
HWES	0.73	0.89	0.80	18
Accuracy		0.77		
Macro Average	0.79	0.77	0.77	35
Weighted Average	0.79	0.77	0.77	35

Table 4.4: Classification Report after adding features

After that we decided more training samples were needed for training and thus added another dataset. This resulted in the following:

	Predicted ARIMA	**Predicted HWES**
Actual ARIMA	26	11
Actual HWES	7	25

Table 4.5: Confusion Matrix and combining two company's stock prices

In this we can see that we get more samples, but the accuracy drops off. This was due to not handling outliers well, along with sub-par feature engineering.

4.2 Best model

After going through all the experiments and trials with various changes throughout, we landed on a multitude of features, on the previous list of features we also added more time based features, along with more data to get more samples. This was the output we noticed in the final iteration. After finding the best features and model hyperparameters, we also tried various different classification algorithms. The findings are shown in the table below.

Class	Precision	Recall	F1-Score	Support
ARIMA	0.79	0.70	0.74	37
HWES	0.69	0.78	0.74	32
Accuracy		0.74		
Macro Average	0.74	0.74	0.74	69
Weighted Average	0.74	0.74	0.74	69

Table 4.6: Classification Report after combining two company's stock prices

	Predicted ARIMA	**Predicted HWES**
Actual ARIMA	146	26
Actual HWES	11	161

Table 4.7: Best Confusion Matrix

Class	Precision	Recall	F1-Score	Support
ARIMA	0.93	0.85	0.89	172
HWES	0.86	0.94	0.90	172
Accuracy		0.89		
Macro Average	0.90	0.89	0.89	344
Weighted Average	0.90	0.89	0.89	344

Table 4.8: Best Classification Report

Model	Test Accuracy	Cross-Validation Accuracy (Average over 5
Naive Bayes	0.63	0.66
Random Forest	**0.89**	**0.85**
Decision Tree	0.85	0.78
SVC	0.68	0.68

Table 4.9: Model Comparison: Training vs Cross-Validation Accuracy

CHAPTER 5: CONCLUSION

This project aimed to provide an improved framework through which model selection can be automated more efficiently and effectively on a dataset containing stock prices. Using Radom Forest Classifier demonstrated better performance than other classification models like SVM, Decision Trees and Naive Bayes. The improvement in model performance was evident through iterative testing. Future work can expand on these models by exploring more complex time series models, and also incorporating additional features and better feature engineering for more robust predictions.

REFERENCES

[1] Talkhi, N., Akhavan Fatemi, N., Jabbari Nooghabi, M. et al. Using meta-learning to recommend an appropriate time-series forecasting model. BMC Public Health 24, 148 (2024). https://doi.org/10.1186/s12889-023-17627-y

[2] Lemke C, Gabrys B. Meta-learning for time series forecasting and forecast combination. Neurocomputing. 2010;73(10):2006–16.

[3] Prudêncio R, Ludermir T. Using machine learning techniques to combine forecasting methods. in Australasian Joint Conference on Artificial Intelligence. 2004. Springer.

[4] Hamilton, J.D. (1994). Time Series Analysis. Princeton University Press.

[5] Enders, W. (2014). Applied Econometric Time Series. Wiley.

[6] Yeo, I. K., Johnson, R.(2000). ”A New Family of Power Transformations to Improve Normality or Symmetry.” Biometrika, 87(4), 954–959.

[7] Hamilton, J.D. (1994). Time Series Analysis. Princeton University Press.

[8] Almasarweh M, Alwadi S. ARIMA model in predicting banking stock market data. Mod Appl Sci. 2018;12(11):4.

[9] De Livera AM, Hyndman RJ, Snyder RD. Forecasting Time Series with Complex Seasonal patterns using exponential smoothing. J Am Stat Assoc. 2011;106(496):1513–27.

[10] Ali AR, Gabrys B, Budka M. Cross-domain Meta-learning for time-series forecasting. Procedia Comput Sci. 2018;126:9–18.